Software Requirement Specification

NumSolve 2.0: A Cross-Platform Intelligent Numerical Analysis Suite for Education and Research

By: Ameer Muqrie Bin Abdul Mutalib

Matric No. : S72433

Supervised by: Dr. Waheed Ali Hussein Mohammed Ghanem

CSF4984 FINAL YEAR PROJECT I

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
UNIVERSITI MALAYSIA TERENGGANU

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

A detailed description of the system to be developed is provided by the Software Requirements Specification (SRS) for NumSolve 2.0. It establishes the system's objectives, scope, functions, and constraints, so that all stakeholders, including users, developers, and evaluators, have a clear understanding. This document describes what the system must do and the conditions under which it must operate, laying the groundwork for design, implementation, and testing. (8allocate , blog author)

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to detail both functional and non-functional requirements for an intelligent, web-based numerical analysis system called NumSolve 2.0. This document serves as a comprehensive guide for developers, project supervisors, and evaluators to ensure that the system design and implementation meet the expectations of its intended users and align with the defined objectives. (Krüger, 2025)

This SRS specifically describes the tasks that the system must complete, the performance and usability requirements that it must meet, and the operational limitations that it will be subject to. Additionally, it creates a shared understanding between all parties involved, reducing uncertainty throughout the design and development stages.

Additionally, as new features or numerical techniques are added, developers may easily update or expand the system thanks to this document, which serves as a platform for future maintenance and improvement initiatives. The SRS guarantees that NumSolve 2.0 is developed methodically, stays user-focused, and achieves its analytical and educational goals by offering a precise, well-defined, and quantifiable specification.

1.2 Scope

NumSolve 2.0 is a web-based system for numerical analysis with features implemented in Java, JSP, and MySQL that helps in carrying out various numerical calculations without the need for advanced or space-consuming tools such as MATLAB. The system provides support for various numerical techniques such as ODE solvers, differentiation, integration, interpolation, etc.

Furthermore, it has a built-in rule-based decision support mechanism that assesses user inputs to suggest a suitable numerical method for a particular problem. This decision mechanism is embedded within the system. It enables numerical computation and assessment. Additionally, it allows for theoretical explanations, visualization of numerical results in a graphical format, report generation, export options, and saving.

To assist users in structured learning, it is proposed that two organizational modules, the Class Management Module and the Quiz Interactive Assessment Module, be developed within the context of the proposed NumSolve 2.0 program. The Class Management Module can assist users in organization, records, and monitoring, whereas the Quiz Interactive Assessment Module can assist users in structured problem-solving activities, utilizing the numerical computation feature of the proposed program, to evaluate user performance.

The students would mostly employ this system to learn and execute tasks, whereas the instructors would employ it to teach and conduct academic analyses. The System Analyst role is also included to manage user access to the system, monitor its performance, analyze its usage, and ensure proper system operation, among other tasks. All this is to ensure the proper functioning and academic integrity of NumSolve 2.0.

1.2.1 Definitions, Acronyms, and Abbreviations

The important terminology, acronyms, and abbreviations used in the **NumSolve 2.0** Software Requirements Specification (SRS) document are defined in this section. These definitions guarantee that readers will consistently comprehend technical and domain-specific terms.

Table 1.1: Definitions, Acronyms, and Abbreviations

Term / Acronym	**Definition**
GUI	Graphical User Interface
CRUD	Create, Read, Update, Delete
IDE	Integrated Development Environment
SRS	Software Requirements Specification
ODE	Ordinary Differential Equation
CSV	Comma-Separated Values
PDF	Portable Document Format
Algorithm	A step-by-step computational procedure used to solve a mathematical or logical problem.
Root-Finding	A numerical method used to determine the value of a variable that makes a function equal to zero.
Interpolation	The process of estimating unknown values between two known data points.
Differentiation	A numerical technique used to compute the rate of change of a function.
Integration	A numerical method for calculating the area under a curve or total accumulated quantity.
Convergence	The tendency of a numerical method to approach a stable solution after several iterations.
Iteration	A single cycle of computation in which a numerical estimate is refined toward the final solution.
Error Tolerance	The acceptable limit of error in a numerical result, usually defined by a small threshold value (e.g., 10^{-6}).
Decision Rule	A predefined logic used by the Rule-Based Decision Support Module to determine the most suitable numerical method.
Visualization	The graphical representation of numerical data or computation results for easier interpretation.

1.3 Overview

The **NumSolve 2.0** system and its needs are thoroughly and systematically described in this publication. It acts as a guide for developers, managers, and assessors to make that the system is planned and executed in line with user requirements and educational goals. This SRS's several sections each support a distinct phase of system comprehension, development, and validation.

The goal, scope, and important terms of the *NumSolve 2.0* project are explained in **Section 1**. The fundamental background information for comprehending the system and the need it fills in numerical calculation and education is given in this part.

The *requirement elicitation techniques* used to get crucial system information are described in **Section 2**. It goes over the sources of requirements, including reference materials and stakeholders, and goes into detail about the different methods, such as brainstorming, interviews, observation, and feedback sessions.

The *system requirements* are described in **Section 3**, which includes both functional and non-functional specifications. While non-functional requirements focus on usability, performance, dependability, and other quality features, functional requirements are further separated according to system users, such as students, educators, and system analysts.

Section 4 is devoted to the system's *requirement analysis*. In addition to a CRUD matrix that shows database operations and entity relationships, it displays the system's modeling using a number of diagrams, such as the use case, activity, class, and sequence diagrams.

A summary of the complete document is given in **Section 5**, which highlights the importance of the requirements and analyses carried out. It concludes that **NumSolve 2.0** can be designed, developed, and deployed successfully because of the explicit and comprehensive requirements that have been stated.

CHAPTER 2

REQUIREMENT ELICITATION TECHNIQUES

The process of obtaining data from stakeholders in order to specify the limitations and functionalities of the system is known as requirement elicitation. In order to make sure the finished system achieves its goals, it aims to determine user needs and expectations. Discussions, observations, and document analysis with students, teachers, and the system analyst were all part of the NumSolve 2.0 process. (Brown, 2025)

2.1 Requirement Sources

The requirements for the **NumSolve 2.0** system were compiled from primary sources, which are listed in this section. Comprehending these sources guarantees that the final system takes into account the technical and educational requirements of its stakeholders. Three primary sources of information were gathered: reference materials, current systems, and stakeholders.

2.1.1 Stakeholders

The main stakeholders involved in this project are as follows:

- **Students:** represent the primary users of the system who require an easy-to-use and efficient tool to carry out and comprehend numerical computations. They look for assistance in choosing suitable numerical techniques, interactive learning aids, and graphical visualization.
- **Educators:** Incorporate researchers and instructors who use the system for instruction and analysis. In order to improve study analysis and teaching efficacy, they place a strong emphasis on correctness, visualization support, and theoretical justifications.

- **System Analyst:** In charge of preserving system dependability, keeping an eye on performance, examining usage data, and controlling user access. Throughout upgrades, the analyst makes that the system stays effective, safe, and functionally consistent.

2.1.2 Documents

To find key algorithms, computational strategies, and theoretical ideas, documents including university course materials and textbooks on numerical analysis were examined. To guarantee academic and technical accuracy, these references served as a guide for the creation and validation of the numerical techniques used in **NumSolve 2.0**.

2.1.3 Existing Systems

To determine their advantages and disadvantages, a number of software programs were examined, including MATLAB, Wolfram Alpha, GeoGebra, Scilab, and the most recent version of NumSolve 1.0. Even while these tools offer sophisticated symbolic and numerical computation skills, they frequently call for a lot of system resources or constant internet access. In order to overcome these drawbacks, **NumSolve 2.0** offers a web-based and platform-independent solution that integrates intelligent learning support, numerical computation, and visualization into a single browser-accessible system.

2.2 Requirement Techniques

The methods for collecting and evaluating the requirements for the creation of the **NumSolve 2.0** system are described in this section. The methods chosen guarantee that, while preserving technical accuracy based on recognized scholarly sources, the data gathered appropriately represents the expectations and difficulties of the intended users. Conversational, observational, and document-centric approaches were the three main methods employed.

2.2.1 Conversational Techniques

To determine their demands and challenges in carrying out numerical computations, interviews and casual conversations were held with a few math students and instructors, including senior lecturers in numerical analysis. Students discussed their difficulties using current tools, including MATLAB, which frequently need extremely complex setup or previous programming experience. Teachers highlighted the value of clear theoretical explanations and visual aids to enhance instruction. The data acquired from these discussions served as a guide for identifying the essential components of the system, such as the graphical visualization, learning support, and rule-based recommendation module.

2.2.2 Observational Techniques

Existing numerical computation platforms, including MATLAB, Wolfram Alpha, GeoGebra, Symbolab, and the previous iteration of the system, NumSolve 1.0, were analyzed during observation sessions. Examining their computational procedures, interface design, and usability was the aim. This finding led to the identification of a number of problems, such as the setup's complexity, reliance on internet access, and the computation parameters' restricted control. In particular, NumSolve 1.0 had few visualization features and no intelligent recommendations. The creation of NumSolve 2.0 was directed by these conclusions in order to guarantee a more intuitive user interface, flexible parameter control, and improved learning support through a web-based platform.

2.2.3 Document-Centric Techniques

To make sure that all of the numerical techniques used in **NumSolve 2.0**—such as bisection, secant, Newton-Raphson, interpolation, differentiation, and integration—follow accurate and standardized mathematical formulas, reference materials, including textbooks and course notes on numerical analysis, were examined. This method allowed the system to function as a computational and instructional tool while guaranteeing academic accuracy and consistency.

CHAPTER 3

SYSTEM REQUIREMENTS

NumSolve 2.0's full set of features and performance requirements are outlined in the System Requirements section. It outlines the functional requirements (what the system must accomplish) and the non-functional requirements (how well the system must fulfill those activities). (GeeksForGeeks, 2025) In order to ensure that all identified user and business needs are converted into precise, workable specifications, this part serves as a link between the requirement elicitation process and the design phase. The design, implementation, and validation of the system are based on the requirements listed here.

3.1 Functional Requirements

Functional requirements outline the essential functions and features that the **NumSolve 2.0** system must have in order to satisfy project and user goals. They detail the roles and interactions of the system's users, namely the **Students**, **Educators**, and the **System Analyst**, and specify what the system should do in particular scenarios.

Every functional requirement is made to guarantee dependability, usability, and accessibility while executing numerical calculations, displaying outcomes, and assisting with academic learning.

3.1.1 Student

The system's main users, who carry out numerical calculations and use it as a learning aid, are students. The following features must be available to students through the system:

- **FR1:** The system shall allow students to register, log in, and manage their profiles securely.
- **FR2:** The system shall allow students to input equations or datasets for numerical computations.
- **FR3:** The system shall perform computations using multiple numerical methods, including root-finding, interpolation, differentiation, integration, and ODE solvers.

- **FR4:** The system shall provide a rule-based decision module to recommend the most suitable numerical method based on user input.
- **FR5:** The system shall display graphical visualizations and step-by-step iterations of the selected numerical methods.
- **FR6:** The system shall allow students to save, view, update, and delete their computation history.
- **FR7:** The system shall allow students to generate and export computation results and reports in PDF or CSV format for reference or submission.
- **FR8:** The system shall allow students to view and access learning materials, including notes, tutorials, worked examples, and multimedia content.
- **FR9:** The system shall allow students to participate in quizzes related to numerical methods and computations.
- **FR10:** The system shall provide gamification elements such as scores, badges, or progress levels to motivate learning.
- **FR11:** The system shall allow students to join and view class groups created by educators.

3.1.2 Educator

Researchers and lecturers who use the method to instruct, mentor, and assess students' comprehension of numerical analysis are considered educators. The following features must be included in the system for educators:

- **FR12:** The system shall allow educators to upload new learning materials, including notes, tutorials, worked examples, or multimedia content.
- **FR13:** The system shall allow educators to update existing learning materials, including file content, title, description, or metadata.
- **FR14:** The system shall allow educators to delete outdated or incorrect learning materials.
- **FR15:** The system shall allow educators to view and access learning materials, just like students.
- **FR16:** The system shall allow educators to create and manage quizzes for students.
- **FR17:** The system shall allow educators to configure gamification rules such as scoring and rewards.

- **FR18:** The system shall allow educators to create, manage, and assign students to classes.

3.1.3 System Analyst

To guarantee stability and dependability, the System Analyst is in charge of maintaining and keeping an eye on the **NumSolve 2.0** system's overall performance. The following features must be included in the system for the system analyst:

- FR19: The system shall allow the system analyst to view system activity logs, user statistics, and performance reports.
- FR20: The system shall allow the system analyst to manage user accounts and access permissions.
- FR21: The system shall allow the system analyst to monitor quiz activity, class usage, and gamification performance.
- FR22: The system shall allow the system analyst to configure and maintain system-wide class and quiz settings.

3.1.4 Summary

Together, these functional requirements make sure that **NumSolve 2.0** assists its intended users, allowing **students** to learn and compute, **educators** to mentor and teach, and the **system analyst** to effectively monitor and maintain system operations.

3.2 Non-Functional Requirements

A collection of specifications known as non-functional requirements (NFRs) outline the **NumSolve 2.0** system's operating characteristics, performance benchmarks, and environmental limitations. These specifications place more emphasis on the system's ability to carry out its intended tasks than on its actual capabilities. They are necessary to maintain high user satisfaction and long-term maintainability while ensuring the program runs effectively, securely, and dependably.

Performance, scalability, portability, compatibility, stability, availability, maintainability, security, localization, and usability are among the most frequently cited non-functional criteria. Each of these prerequisites enhances the system's overall quality and guarantees that **NumSolve 2.0** may be utilized successfully in a range of computational and academic settings.

- **NFR1: Performance and Scalability**
  The system should be able to perform basic numerical computations like root solving, differentiation, and interpolation in **3 seconds** for most input sizes. More complex numerical computations like solving an ODE or large data sets should take **5 seconds**. The graphics should render in **2 seconds** once the computation is finished. The architecture should be scalable, so new numerical services or new users can be added without affecting the performance of the system.

- **NFR2: Portability and Compatibility**
  The system shall be developed in the form of **web-based software** that can be accessed using common web browsers like Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. The system shall be compatible with common operating systems like Windows, macOS, and Linux. The system shall run without any need to install or rely on any external computational platform like MATLAB or Python.

- **NFR3: Reliability, Maintainability, and Availability**
  The system shall function in a reliable manner and to a large extent without failure and shall have at least **99%** uptime during normal academic use times. The system shall ensure that all the results obtained in numbers shall be similar and shall be reproducible when given similar parameters. The system source code shall ensure that there is ease of understanding and modification.

- **NFR4: Security**
  The system will employ strong authentication and authorization. This will ensure that the accounts and data of the users are secure. The passwords of the users will be protected using secure hashing techniques. The sensitive data will be protected from unauthorized access, modification, or deletion. The input validation and proper session management will be implemented to overcome the common security threats.

- **NFR5: Localization and Compatibility with Local Context**
  The system is set to use English in its interface. However, in the future, other languages may be supported. The numbers, reports, and data exports will all be in accordance with global academic standards to ensure compatibility between the regions and the global aspects in education and research.

- **NFR6: Usability and User Interface Design**
  The system's interface must be intuitive and user-friendly, whether you're a beginner or a long-term user. Users should be able to do everyday activities, for example, typing out equations and calculating results with a minimal instruction manual. This is made possible by good layout, the assistance of tooltips, and validation information. Lastly, the user interface must be created through the usage of JSP, HTML, and CSS, and must be responsive so that the system runs properly on different displays.

- **NFR7: Data Integrity and Accuracy**
  The calculations being performed should be done through tried-and-tested algorithms to ensure accuracy of results. Data consistency throughout the calculation, storage of the data, and creation of the report needs to be maintained by the system. Error handling needs to be implemented to ensure that no damage to data occurs. No incorrect outcomes of calculation should also occur.

These non-functional requirements guarantee that **NumSolve 2.0** functions as a reliable, secure, and effective learning platform that satisfies contemporary academic and usability standards in addition to being a useful and intelligent computational tool.

CHAPTER 4

REQUIREMENT ANALYSIS

Requirements analysis documents on the needs of the new system. By collecting feedback from users, educators, and analysts it guarantees that the system satisfies the needs of stakeholders. This structuring gives clarity to the system design, reduces vagueness and serves as guide for development of the use-case diagrams and the detailed specifications.

4.1 Use Case Diagram

Use-case diagrams offer a broad view of the system, showing the interactions between users (actors) and the operation of the system (use cases). They provide a powerful way to see the extent and limits of the system, so stakeholders can understand how the system will support business processes. Fig.1 depicts the main use case of NumSolve 2.0 and its stakeholders. (Archimetric, 2025)

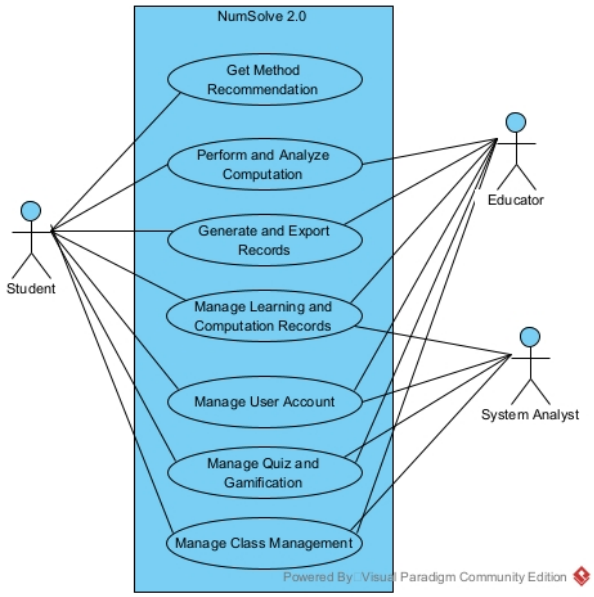

Figure 4.1: Use-Case Diagram of NumSolve 2.0

4.2 Use Case Descriptions

There is an explanation for each use-case that indicates who or what started-up, normal and alternate flows whe re applicable, and responses of the system. These descriptions are prescriptive of the system, and serve as a basis for system development, testing and verification.

Table 4.1: Use Case 1: Manage User Account

Use Case Name: Manage User Account	**ID:** UC1	**Importance Level:** High
Primary Actor: Student, Educator, System Analyst	**Use Case Type:** Primary, Essential	
Stakeholders and Interests: Students and educators manage personal accounts. System Analyst ensures account integrity and security. **Brief Description:** Users can register, log in, and update account details. The System Analyst manages and maintains user accounts. **Trigger:** User wants to register or log in. **Type:** External **Normal Flow of Events:** 1. User access the system through web browser. 2. System displays authentication page. 3. User enters credentials or registration info. 4. System validates and grants access or creates account. 5. User updates profile if needed. 6. System Analyst may manage, modify, or delete accounts. **Sub-Flows:** Password reset via email if user forgets credentials. **Alternate/Exceptional Flows:** Invalid credentials prompt an error and re-entry.		

Table 4.2: Use Case 2: Get Method Recommendation

Use Case Name: Get Method Recommendation	**ID:** UC2	**Importance Level:** High

Primary Actor: Student, Educator **Use Case Type:** Primary, Essential
Stakeholders and Interests:
Users want guidance on selecting the most suitable numerical method for their problem.
Brief Description: System analyzes input equations or datasets and suggests optimal numerical methods with explanations.
Trigger: User submits problem or dataset for recommendation.
Type: External
Normal Flow of Events:

1. User selects recommendation option.
2. Input is validated by the system.
3. Rule-based module analyzes the problem.
4. Recommended method(s) are displayed with rationale.

Sub-Flows:
System ranks multiple suitable methods and provides brief explanations.
Alternate/Exceptional Flows:
Invalid input format triggers an error and requests correction.

Table 4.3: Use Case 3: Perform and Analyze Computation

Use Case Name: Perform and Analyze Computation	**ID:** UC3	**Importance Level:** High

Primary Actor: Student, Educator **Use Case Type:** Primary, Essential
Stakeholders and Interests:
Users perform calculations using selected numerical methods and view detailed results.
Brief Description: Run numerical computations and display outputs with graphs and stepwise breakdown.
Trigger: User chooses method and provides input data.
Type: External
Normal Flow of Events:

1. User selects numerical method.
2. Input parameters are provided.
3. System executes computation.
4. Results and graphs are displayed.
5. User may save or export results.

Sub-Flows:
Compare outputs across different methods for accuracy or performance.
Alternate/Exceptional Flows:
Invalid inputs cause error messages with correction suggestions.

Table 4.4: Use Case 4: Manage Learning and Computation Records

Use Case Name: Manage Learning and Computation Records **ID:** UC4 **Importance Level:** Medium

Primary Actor: Student, Educator, System Analyst **Use Case Type:** Primary, Essential

Stakeholders and Interests:
Users manage previous computations and teaching resources. System Analyst ensures database integrity.

Brief Description: Create, update, or delete stored computations and learning materials.

Trigger: User accesses the records section.

Type: External

Normal Flow of Events:

1. User opens records section.
2. System displays available materials.
3. User selects record to view or edit.
4. Educator may upload or update materials.
5. System updates database.

Sub-Flows:
Categorize materials; system provides usage statistics.

Alternate/Exceptional Flows:
File upload failure triggers retry prompt.

Table 4.5: Use Case 5: Generate and Export Records

Use Case Name: Generate and Export Records	**ID:** UC5	**Importance Level:** Medium

Primary Actor: Student, Educator, System Analyst
Use Case Type: Primary, Essential

Stakeholders and Interests:
Students and educators generate computation records for learning, submission, or analysis purposes, while the system analyst generates summary records for monitoring and evaluation.

Brief Description: Allows users to generate, view, and export computation and learning records in PDF or CSV format.

Trigger: User selects a computation record or dataset for export.

Type: External

Normal Flow of Events:

1. User navigates to the records or reports section.
2. User selects a computation or learning record.
3. System compiles the selected data.
4. User chooses the desired export format.
5. System generates and saves the exported file.

Sub-Flows:
System may include graphs, iteration details, and comparative analysis in the exported records.

Alternate/Exceptional Flows:
If data retrieval fails, the system displays an error message and prompts the user to retry.

Table 4.6: Use Case 6: Manage Quiz and Gamification

Use Case Name: Manage Quiz and Gamification	**ID:** UC6	**Importance Level:** Medium

Primary Actor: Student, Educator **Use Case Type:** Primary, Essential
Stakeholders and Interests:
Educators require tools to assess student understanding through quizzes, while students benefit from gamification features that enhance motivation and engagement in learning numerical methods.
Brief Description: Allows educators to create and manage quizzes and gamification settings, while students participate in quizzes and track their learning progress.
Trigger: User accesses the quiz or gamification module.
Type: External
Normal Flow of Events:

1. Educator creates or updates a quiz.
2. System stores quiz configuration.
3. Student attempts the quiz.
4. System evaluates submitted answers.
5. System updates scores, badges, or progress indicators.

Sub-Flows:
Gamification rules may assign rewards based on quiz performance.
Alternate/Exceptional Flows:
Invalid quiz configuration or submission triggers an error message and correction prompt.

Table 4.7: Use Case 7: Manage Class Management

Use Case Name: Manage Class Management **ID:** UC7 **Importance Level:** Medium
Primary Actor: Educator, Student, System Analyst **Use Case Type:** Primary, Essential **Stakeholders and Interests:** Educators require structured class management for teaching and monitoring students, students need access to assigned classes, and the system analyst ensures class data integrity and proper system usage. **Brief Description:** Enables educators to create and manage classes, students to join or view assigned classes, and system analysts to oversee class structures. **Trigger:** User accesses the class management module. **Type:** External **Normal Flow of Events:** 1. Educator creates a new class. 2. Students join or are assigned to the class. 3. System stores and updates class information. 4. Users view class-related content and records. **Sub-Flows:** System analyst may monitor class activity and usage statistics. **Alternate/Exceptional Flows:** Duplicate class name or invalid enrollment triggers an error message and retry request.

4.3 Activity Diagrams

The activity diagrams specify detailed flows of the system's major activities in NumSolve 2.0. Each diagram shows how the user (student, teacher and system analyst) interacts with the system and is depicted as a sequence of process steps with decision points and alternative flows. These graphs give cleat intuition on What goes inside Account Management,Method Recommendation, Computation,Record Handling and Report Generation.

4.3.1 Activity Diagram 1: Manage User Account

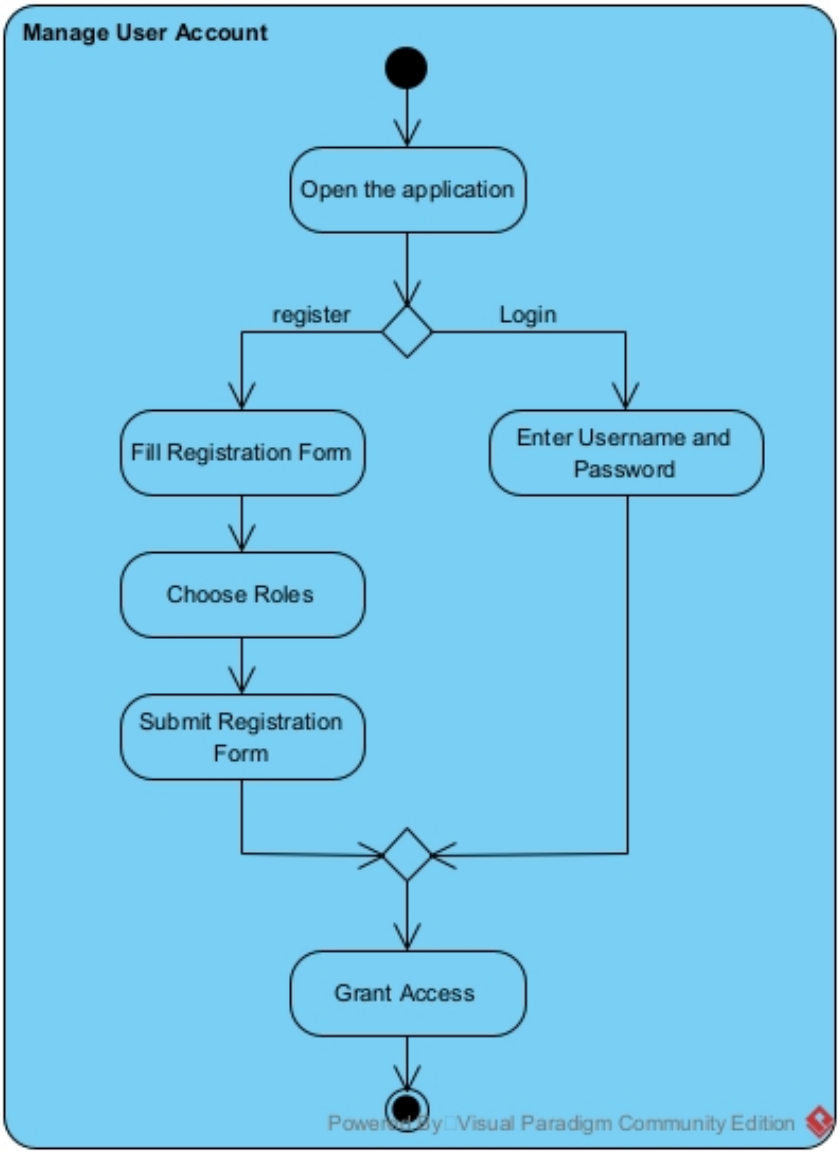

Figure 4.2: Activity Diagram for Manage User Account

Description: This diagram outlines how users interact with the system to create, access, or manage their personal accounts in NumSolve 2.0. The process begins when the user access the system through web browser and the user selects either login if have entered the app before or registration if first time using the app. The system checks the entered details and determines whether access should be permitted or not.

4.3.2 Activity Diagram 2: Get Method Recommendation

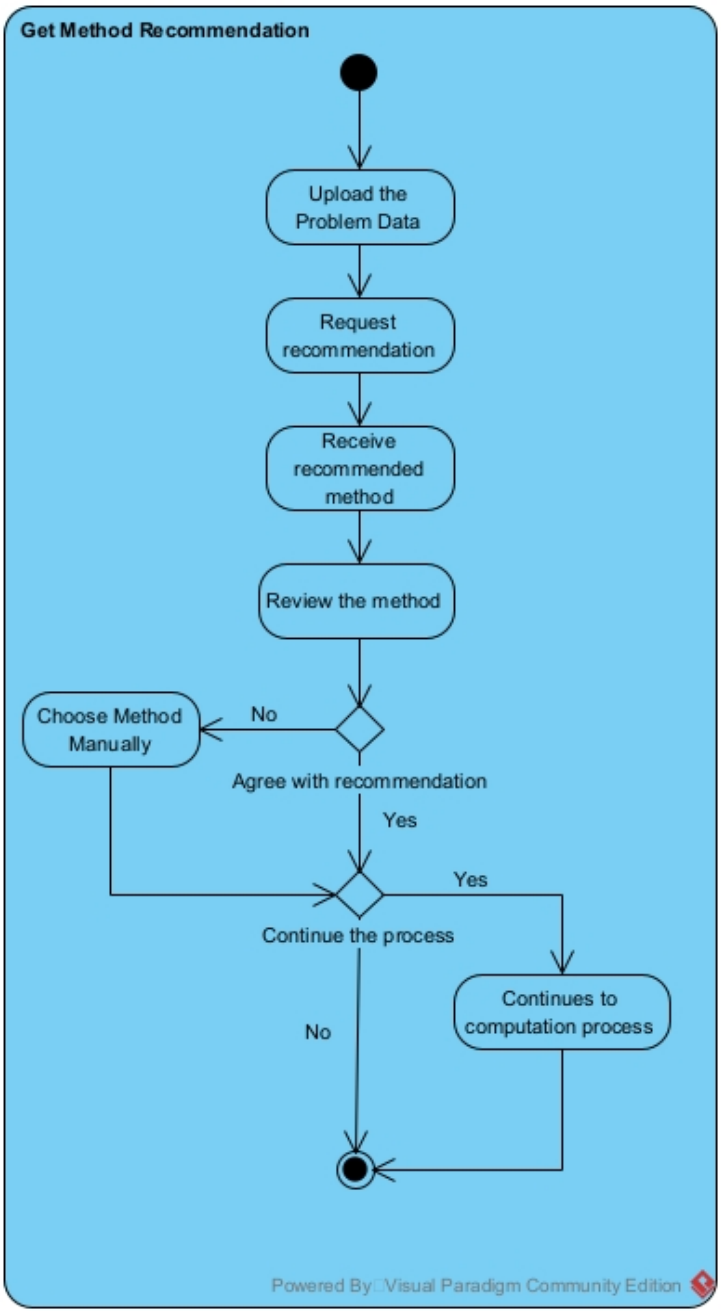

Figure 4.3: Activity Diagram for Get Method Recommendation

Description: This Diagram show how system provide guidance to choose the best method. The user will submit or upload the equation which will be evaluated by the system. After been evaluated, the system will recommend the best method to be used and we can choose to follow the recommendation or not. If no, then we can choose the method manually but if we agree then we can continue the computation process or not.

4.3.3 Activity Diagram 3: Perform and Analyze Computation

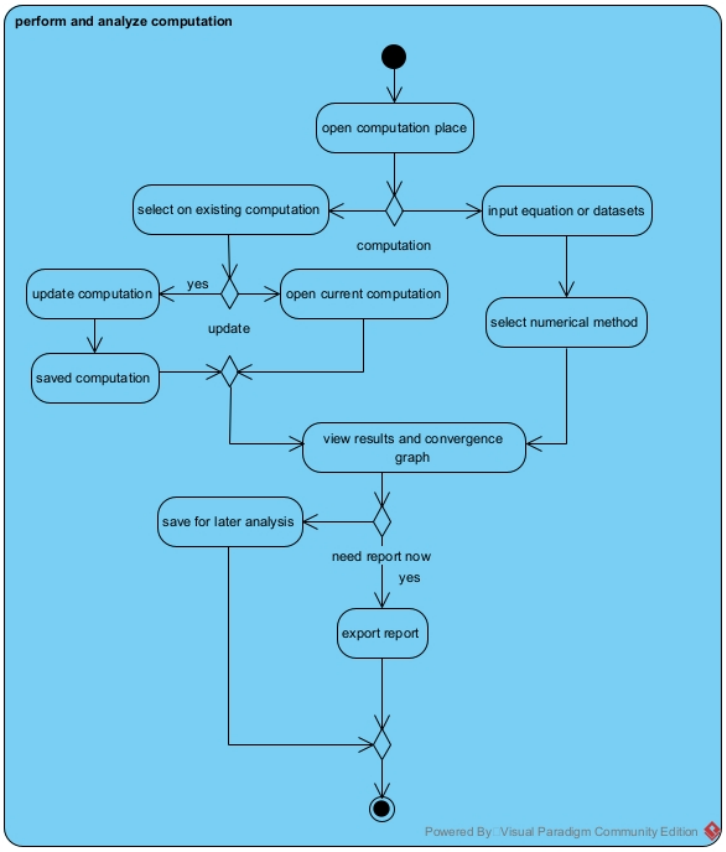

Figure 4.4: Activity Diagram for Perform and Analyze Computation

Description: The diagram show the Computation process. The user firstly must enter equation or datasets or they can open the computation that have been made. For both they can update if want to change the recorded one or to make new one. Then after that they must choose the numerical method that they want to use. After that, the system will calculate and review the answer and convergence graph. The user can choose for detail information or not. If yes then it will show the step by step iterations to get extra knowledge. The user also can export the report or if dont want to export immedietly then they can save for later analysis use.

4.3.4 Activity Diagram 4: Manage Learning Materials

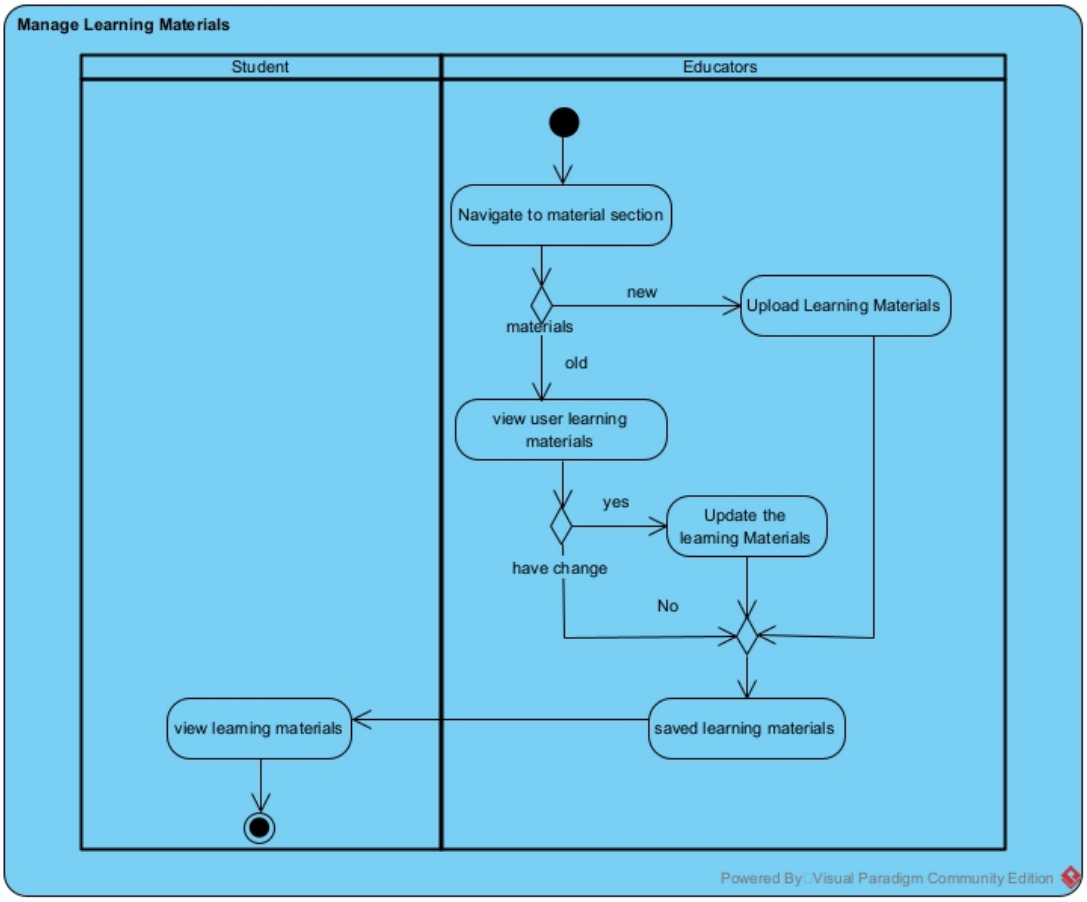

Figure 4.5: Activity Diagram for Manage Learning Materials

Description: This diagram show workflow for managing learning materials. Educators and student can open the materials section. For student they can only search the materials that they want and the system will show the materials they wanted. For Educators they also can search the materials and they can create their own materials. If Educators want, they can upload their materials to the system and saved it or they can update their materials also.

4.3.5 Activity Diagram 5: Generate and Export Reports

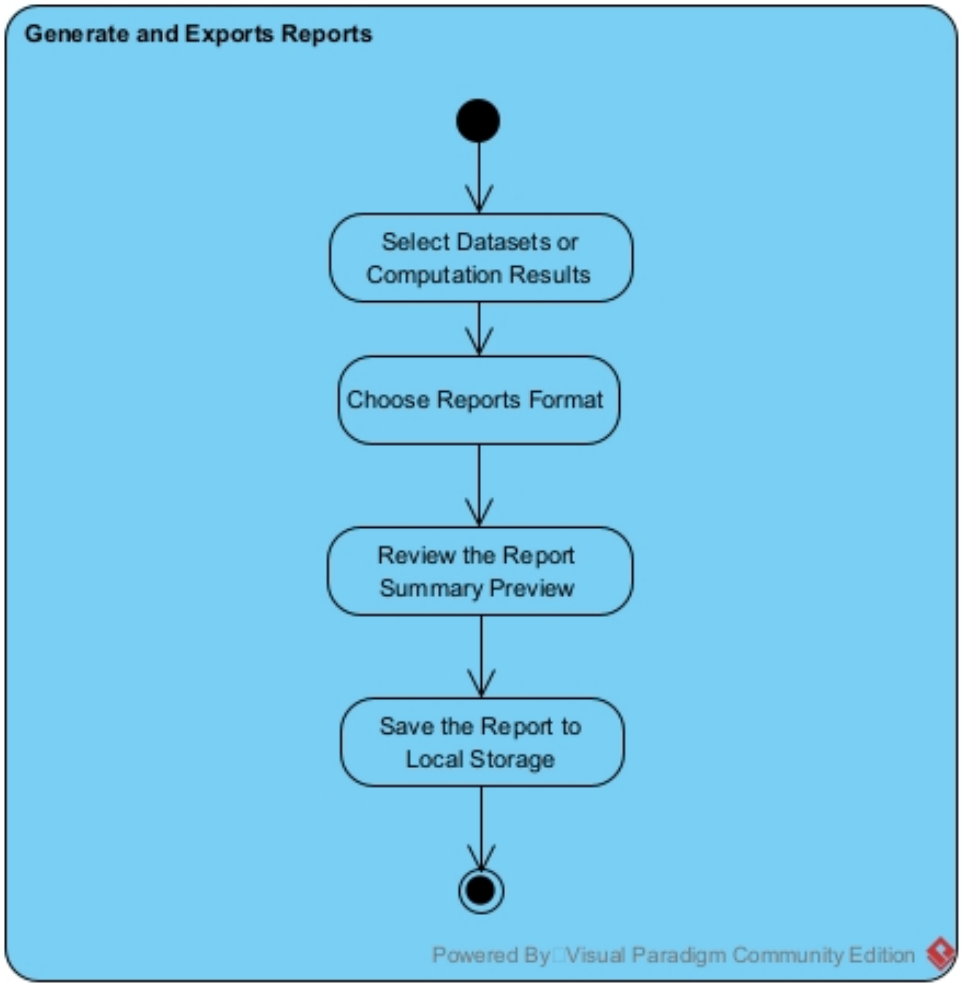

Figure 4.6: Activity Diagram for Generate and Export Reports

Description: This diagram shows the process of generate and export reports. The user can select datasets or computation records that they want to export. Then the user need to choose type of report format between CSV and PDF. Before exporting they can review the report summary to make sure they approved it. Then after approved it, they can saved the report to the local storage on their gadget.

4.3.6 Activity Diagram 6: Manage Quiz and Gamification

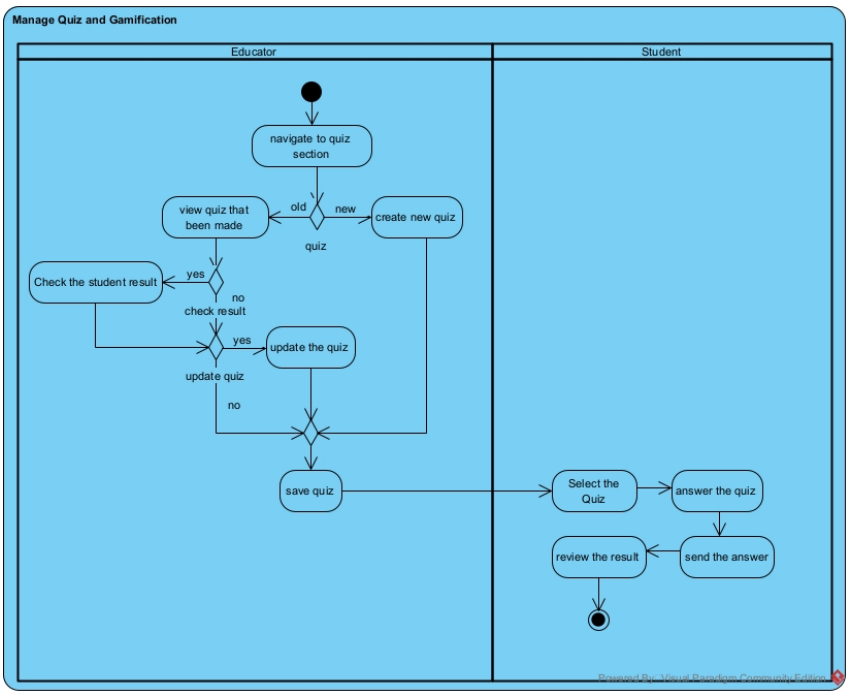

Figure 4.7: Activity Diagram for Manage Quiz and Gamification

Description: This diagram illustrates the management and attempt of the quizzes. The Educator will start by looking at the list of quizzes. The Educator can then choose either to create a new quiz or edit an existing one. The Educator can then edit the questions or settings and even look at the scores of the students. The Educator then clicks 'save' after finishing. The Student will then start by selecting the title of the quiz. The Student will then attempt the questions and click 'submit' after finishing. The system will then calculate and display the result with a badge for the Student.

4.3.7 Activity Diagram 7: Manage Class management

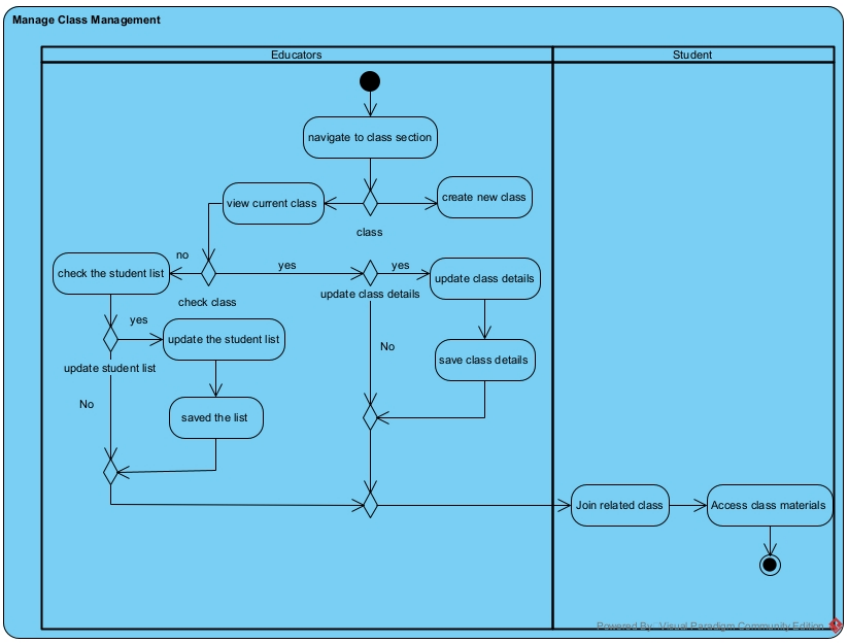

Figure 4.8: Activity Diagram for Manage Class management

Description: The diagram above illustrates the process of class info management, which includes student enrollment in the class by the Educator. The Educator starts by viewing the classes currently available, which then gives the option to create a new class or edit the details of an existing class. Within the edit class details option, the Educator can manage the class by assigning new students to the class or removing students who are currently assigned to the class. After completing the above options, the class info is then saved by the Educator. The role of the Student is to request to be assigned to a class, which then gives the student the option to view the class details.

4.4 Class Diagram

The class diagram for NumSolve 2.0 offers a static perspective to the system, including the key classes, their properties, methods, and interactions between the various classes. The diagram represents how students, teachers, and system analysts communicate with other components including User Account, Computation, Method Recommendation, Records, and Reports. The class diagram assists in appreciating the system structure as well as implementing the system functions. (Archimetric, 2025)

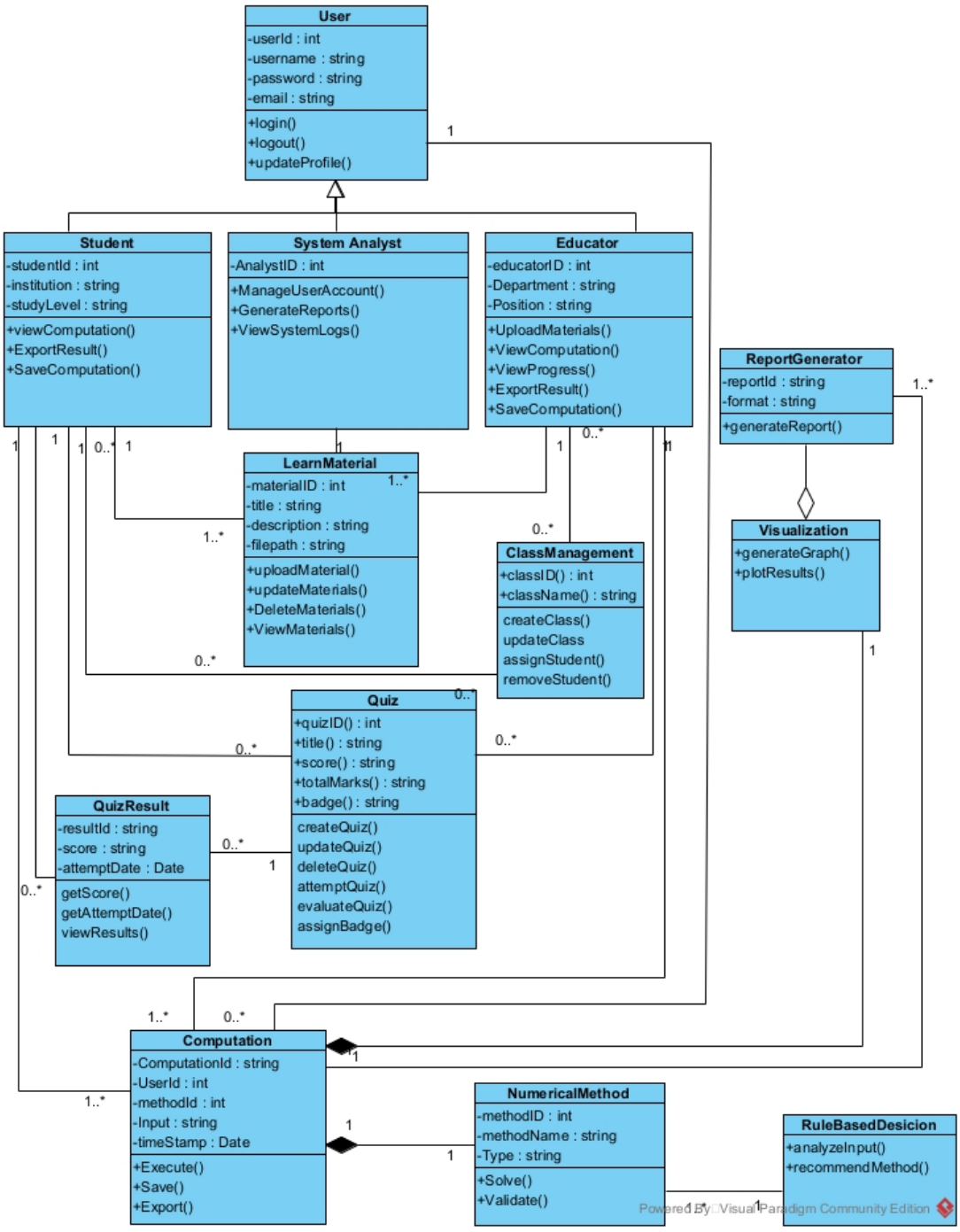

Figure 4.9: Class Diagram of NumSolve 2.0

Description: The class diagram above gives an overview of the system structure as well as the relationships between the main components of the system. The main class in this system is the User class, which contains basic information needed to identify users. This class is then extended into three roles: Student, Educator, and System Analyst.

The Student class enables participation in quizzes, viewing of computational results, saving of results, and keeping track of student records. The Educator class oversees learning activities that include class creation, class management, learning material upload, quizzes, and student tracking. The System Analyst class oversees activities such as account management, report generation, and log files.

With the following classes, the educational materials are managed: ClassManagement, LearnMaterial, and Quiz. LearnMaterial takes care of the learning materials, ClassManagement deals with class creation and assignment of students to classes, and Quiz takes care of quiz creation and evaluation using QuizResult.

The role of the Computation class is to handle all the computational processes. It will store all the inputted data, method selection, and computation results. The Computation class interacts mainly with the NumericalMethod class, which describes the methods for solving mathematical equations, and the RuleBasedDecision class, which recommends the best method based on user input.

Finally, the ReportGenerator and Visualization classes provide structured reporting and a graphical representation of data, enabling easy interpretation of computational and performance data.Overall, the above diagram clearly indicates a comprehensive view of user management, educational resources, computation, and reporting within the system.

4.5 Sequence Diagrams

Sequence diagrams for NumSolve 2.0 illustrate the interactions between actors and the system for each main functionality. They show the sequence of messages exchanged between objects, highlighting the order of operations, decision points, and system responses. These diagrams complement the use-case and activity diagrams by providing a dynamic view of the system behavior during each process. (Archimetric, 2025)

4.5.1 Sequence Diagram 1: Manage User Account

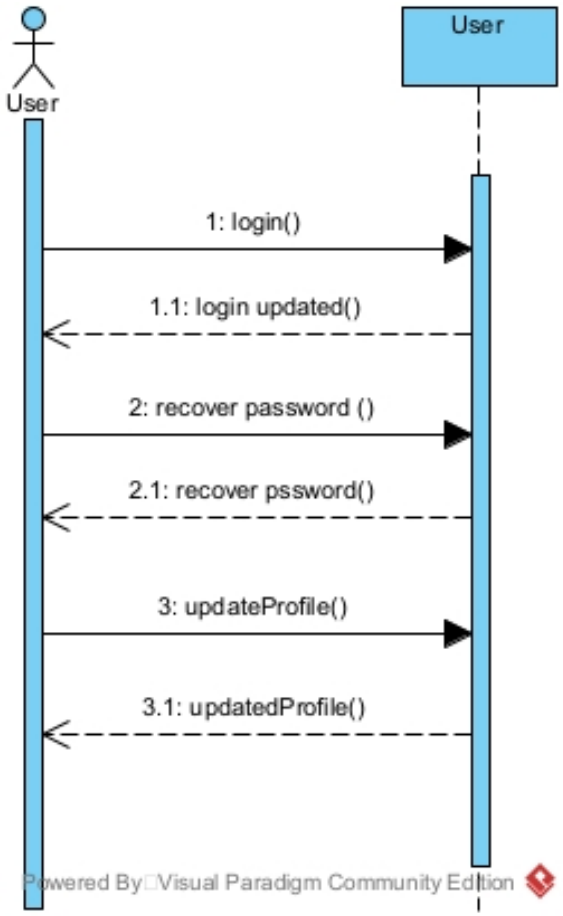

Figure 4.10: Sequence Diagram for Manage User Account

Description: diagram represents the interaction of a user, either a student, educator, or system analyst, with the system that manages the user's account. The user first opens the system, adds personal details, a role, and submits the registration form. The system then checks that the user has registered by confirming that the registration is complete. The user can then log in to the system by entering a password and a user name. The user can forget the password, so they can recover it by executing the recoverPassword() function. The user can also change some of the personal details by executing the updateProfile() function, upon which the system updates the profile in the account.

4.5.2 Sequence Diagram 2: Get Method Recommendation

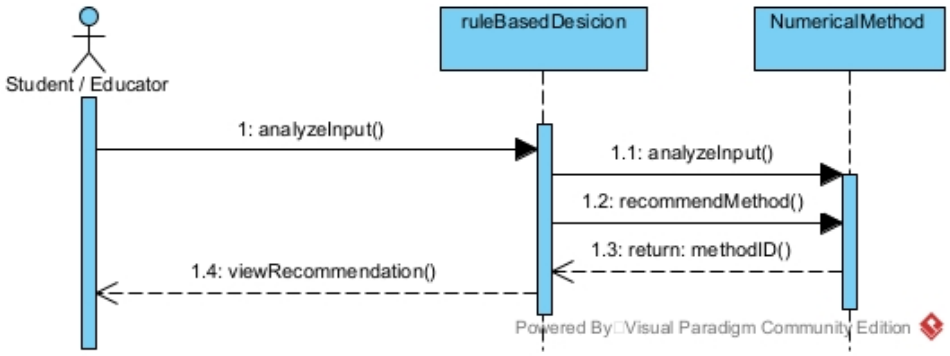

Figure 4.11: Sequence Diagram for Get Method Recommendation

Description: The following sequence diagram illustrates the process involved in the Rule Based Decision System. The Student or Educator initiates the process by sending a query to analyze the equation string using the analyzeInput() function. The RuleBasedDecision object examines this query and consults the NumericalMethod class to examine the properties of the equation. The most appropriate numerical method is determined using the recommendMethod() function, and the Method ID is returned to the user.

4.5.3 Sequence Diagram 3: Perform and Analyze Computation

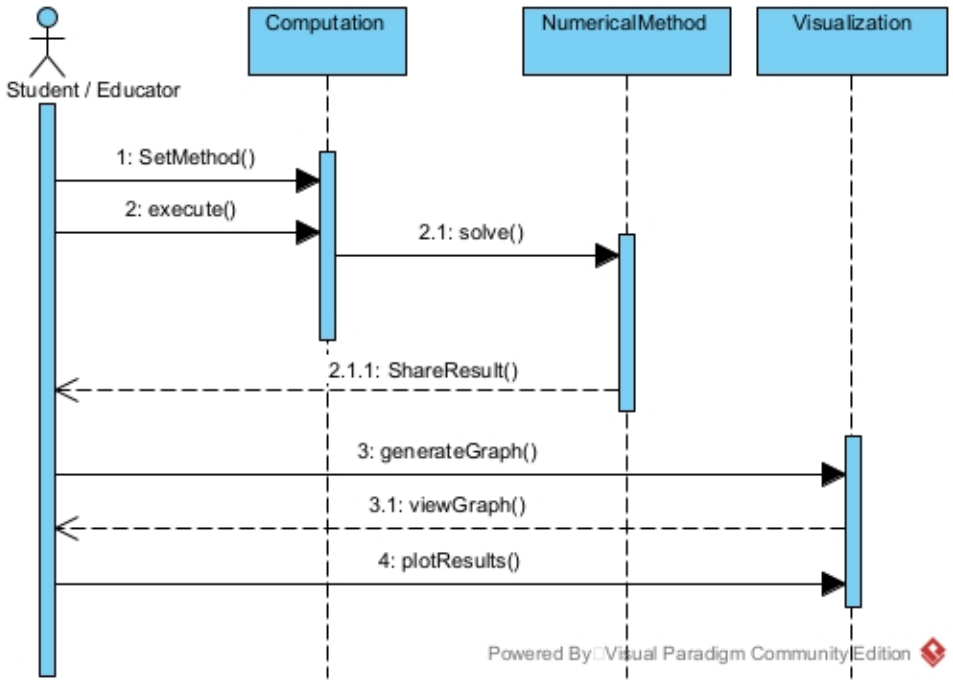

Figure 4.12: Sequence Diagram for Perform and Analyze Computation

Description: This is a description of a process of performing a numerical computation. As shown in the figure, a user is first required to input a method ID and the appropriate input data. At this point, the Computation object initiates the Execute command, which in turn invokes the Solve method of the class named NumericalMethod. After computation, the user is then allowed to view the results. Using the class named Visualization, the user is able to create a graph of the results, as well as a convergence chart, using the methods named generateGraph and plotResults, respectively.

4.5.4 Sequence Diagram 4: Manage Learning Materials

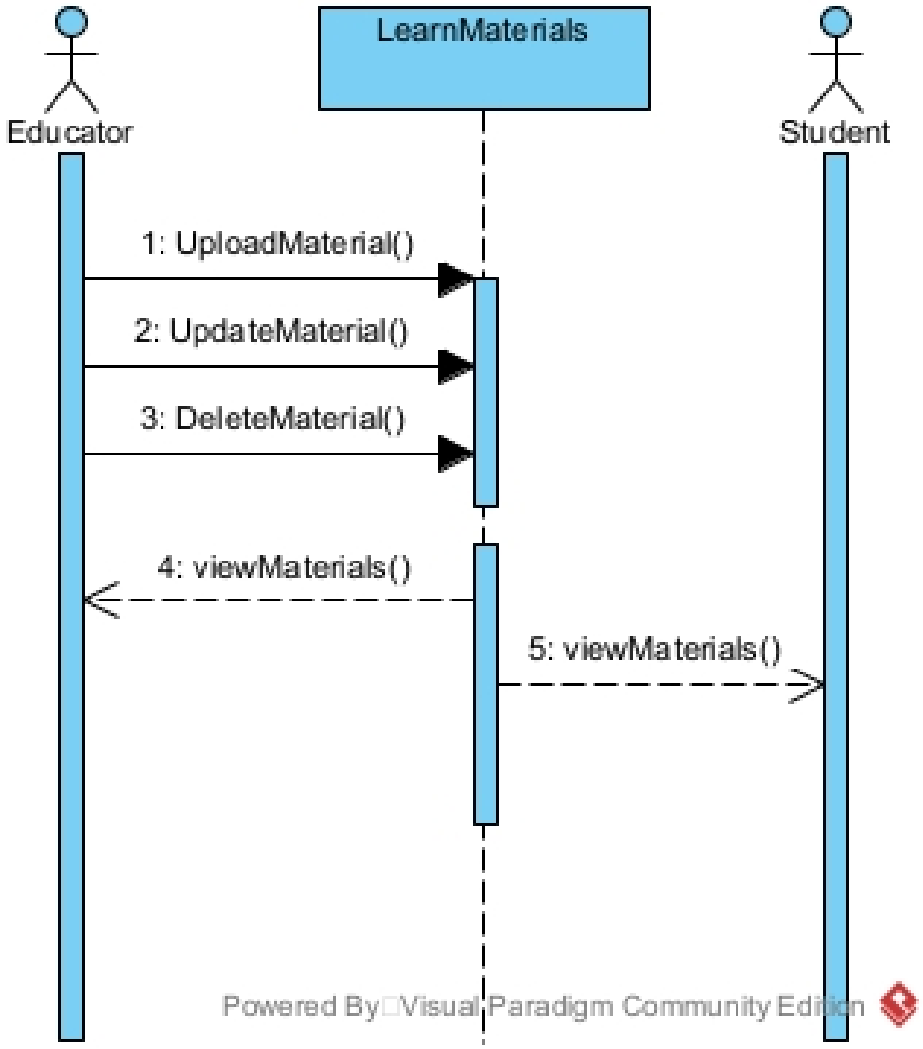

Figure 4.13: Sequence Diagram for Manage Learning materials

Description: This is a sequence diagram showing management of educational materials/content. Here, an Educator sends an uploadMaterial() or updateMaterials() request in order to upload new materials or update existing ones, respectively. This is then acknowledged by the system. There is also an action where an Educator can delete materials/content using the DeleteMaterials() function. Both an Educator and Student can access materials/content, as depicted in this diagram where they both request ViewMaterials().

4.5.5 Sequence Diagram 5: Generate and Export Reports

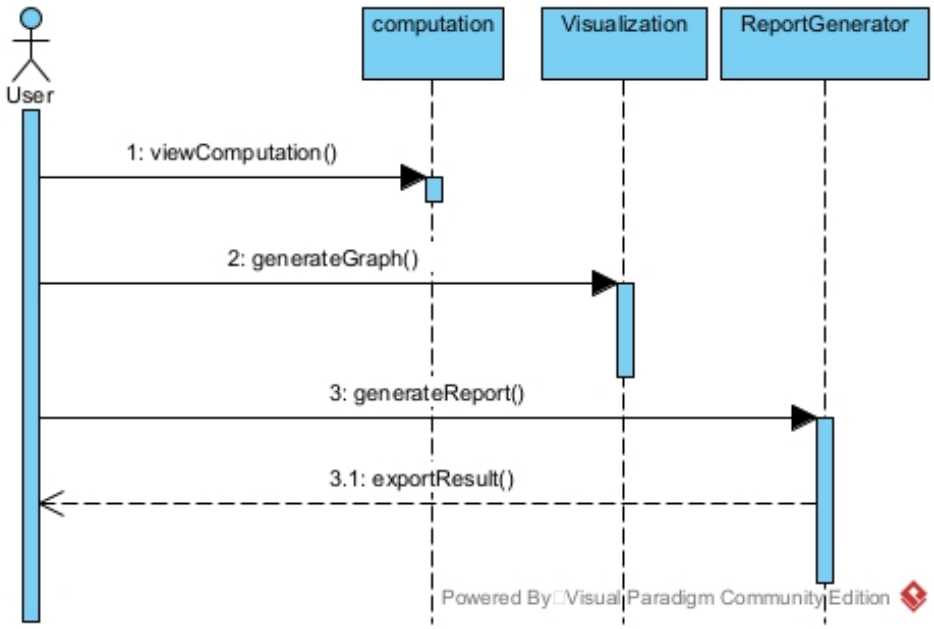

Figure 4.14: Sequence Diagram for Generate and Export Reports

Description: This is a diagram representing the process involved in generating and exporting reports from saved data. A user begins by retrieving a saved data record from the system using the viewComputation method. They then proceed to request a visual summary, thus activating the generateGraph method in the Visualization class. To obtain a formal report, the user is required to specify their preferred format before proceeding to request the generation of a report through the ReportGenerator class. They then proceed to request an export of the data in their preferred format through the ExportResult method, after which it is exported by the Computation class.

4.5.6 Sequence Diagram 6: Manage Quiz and Gamification

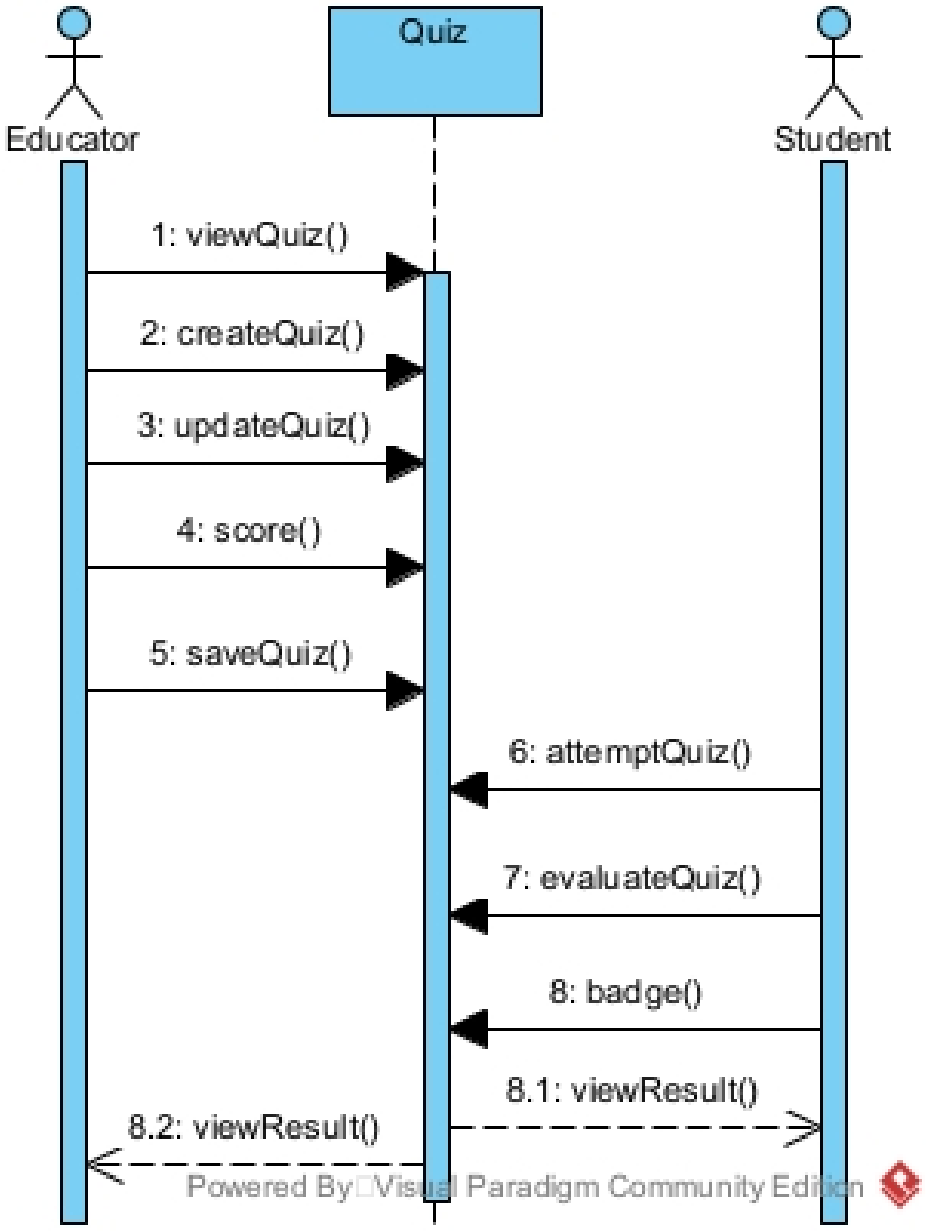

Figure 4.15: Sequence Diagram for Manage Quiz and Gamification

Description: illustrates the interactions for quiz management and participation, which are performed by the Educator and the Student, respectively. The Educator will first invoke the function viewQuiz() to display the quiz list, then createQuiz() to create a new quiz, updateQuiz() to update an existing quiz, and score() to monitor student performance before saving the quiz changes through the function saveQuiz(). Meanwhile, the student will select a quiz title and invoke the function attemptQuiz() to participate in the quiz, which will then trigger the function evaluateQuiz() to evaluate the quiz result, providing a badge for the student if the achievement criteria are met.

4.5.7 Sequence Diagram 7: Manage Class Management

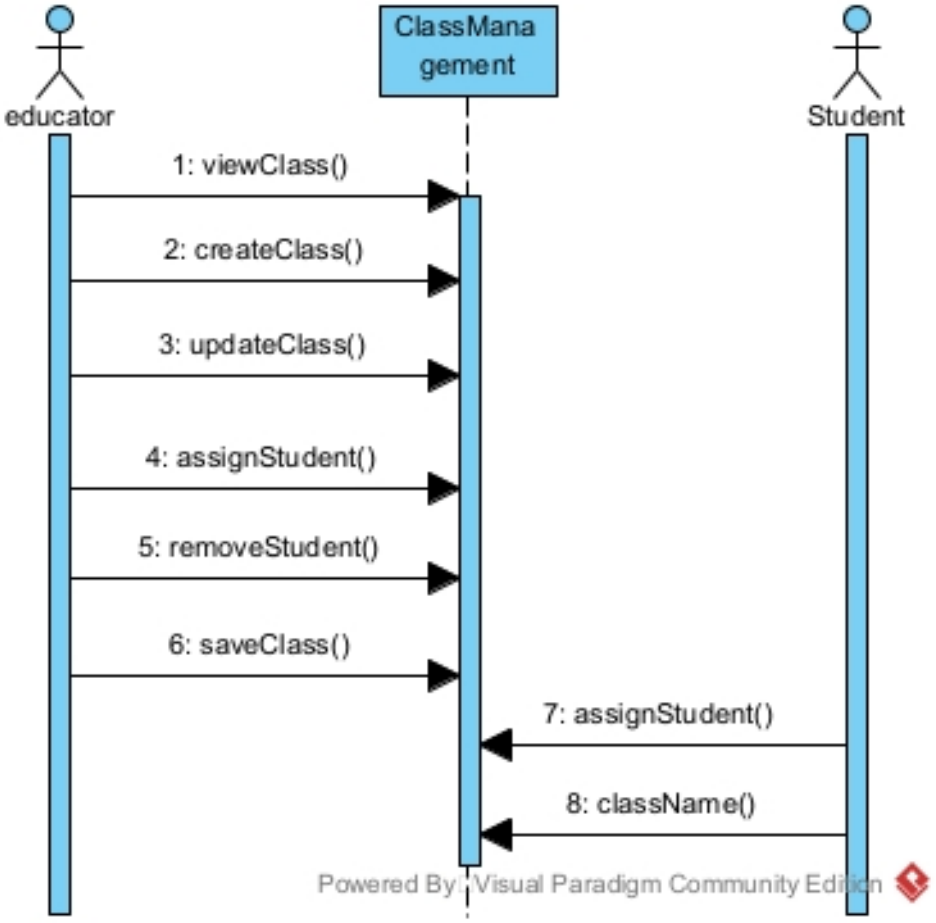

Figure 4.16: Sequence Diagram for Manage Class Management

Description: The diagram depicts a process flow on how classes are organized. The Educator initiates by checking the classes through viewClass(). The Educator can add a class through createClass() or edit a class through updateClass(). While updating a class, the Educator can manage the class by adding members through assignStudent() or deleting members through removeStudent(). The Educator then updates the class through saveClass(). The Student will be able to join a class through assignStudent() and can view the class through className().

4.6 CRUD Matrix

The CRUD Matrix shows how various actors are able to create, read, update, or delete the primary system entities present in NumSolve 2.0. The significance of this matrix is that all roles, whether students, teachers, or system analysts, are identified, making all functions related to secured access and workflows possible through this diagram alone. (Hoogenraad, 2023)

Table 4.8: CRUD Matrix of NumSolve 2.0

Entity	Actions	C	R	U	D
User	Student and Educator register new accounts	x			
	Student and Educator view their profiles		x		
	Student and Educator update their profiles (and password)			x	
	System Analyst manages user accounts	x	x	x	x
Computation	Student and Educator execute new computation	x			
	Student and Educator view saved computations		x		
	Student and Educator export/save results			x	
	Student and Educator delete their computation records				x
Learn Material	Educator uploads new learning material	x			
	Student and Educator view learning materials		x		
	Educator updates existing learning materials			x	
	Educator deletes outdated learning materials				x
Quiz	Educator creates new quizzes	x			
	Student attempts quizzes (Read Questions)		x		
	Educator views quiz list and student scores		x		
	Educator updates quiz details			x	
	Educator deletes quizzes				x
Class Mgmt	Educator creates new classes	x			
	Student and Educator view class details		x		
	Educator updates class (assign/remove student)			x	
	Student joins class (Updates student list)			x	
	Educator deletes a class				x
Report	Student and Educator generate computation reports	x			
	Student, Educator, and Analyst view generated reports		x		
	System Analyst updates system-wide reports			x	
	System Analyst deletes reports				x
Note: C=Create, R=Read, U=Update, D=Delete					

CHAPTER 5

SUMMARY

The SRS document provides a well-defined, structured representation of the NumSolve 2.0 system, whereby all key behaviors, restrictions, and expectations are established before development starts. Every part of the system, whether decision support, numerical processing, user access, or result handling—has been decomposed by Diagrams, descriptions, and statements of requirements to remove ambiguity and guide accurate implementation. The presence of roles, interaction, and system regulation guarantees that both functional requirements and educational integrity are maintained.

Taken altogether, NumSolve 2.0 can be seen as a useful, well-structured solution to that enables numerical learning beyond simple calculations. It combines instruction, visualization, result management, and user accessibility are incorporated as a unitary environment. Having well-defined requirements, the project is ready to proceed into the stage involving design and development, which is well tied to academic requirements, usability, expectations, and system reliability.

REFERENCES

8allocate (blog author) (2023). The guide to writing software requirements specification. *8allocate Blog*.

Archimetric (2025). Using use case, class, and sequence diagrams. *Archimetric Guide*.

Brown, W. (2025). Requirement elicitation techniques: A guide for business analysts. *The Knowledge Academy Blog*. Accessed Nov 13, 2025.

GeeksForGeeks (2025). Functional and non-functional requirements. *GeeksForGeeks*.

Hoogenraad, W. (2023). The crud matrix explained. *ITpedia*. Accessed Nov 13, 2025.

Krüger, G. (2025). How to write a software requirements specification (srs) document. *Application Lifecycle Management, Software Quality*.

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **102** Not Cited or Quoted 15%
  Matches with neither in-text citation nor quotation marks
- **8** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
1% Publications
15% Submitted works (Student Papers)